4-BIT ALU DESIGN USING DSCH AND VERILOG HDL

Professional Project Report

Prepared by: Manoj Goud

Abstract

This project implements a 4-bit Arithmetic Logic Unit (ALU) using DSCH and Verilog HDL. The ALU supports Addition, Subtraction, AND, OR and XOR operations through modular design.

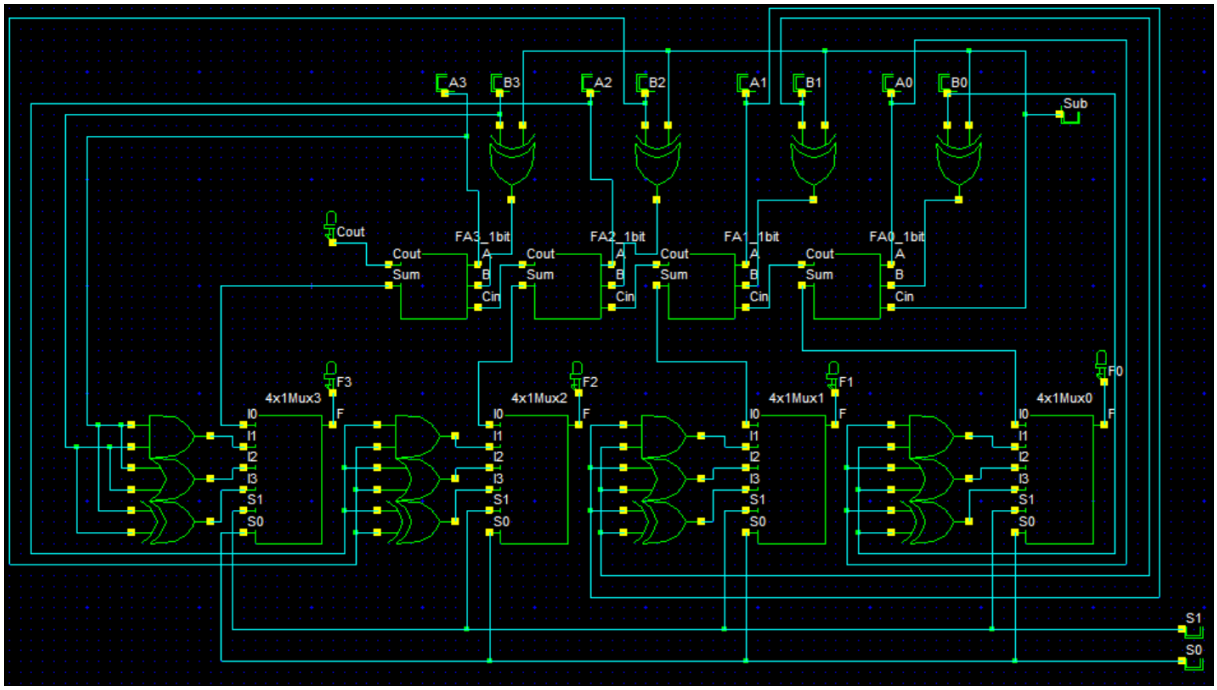
Objectives

Design a reusable Full Adder, implement a 4-bit ALU, verify operation selection through multiplexers and generate Verilog HDL.

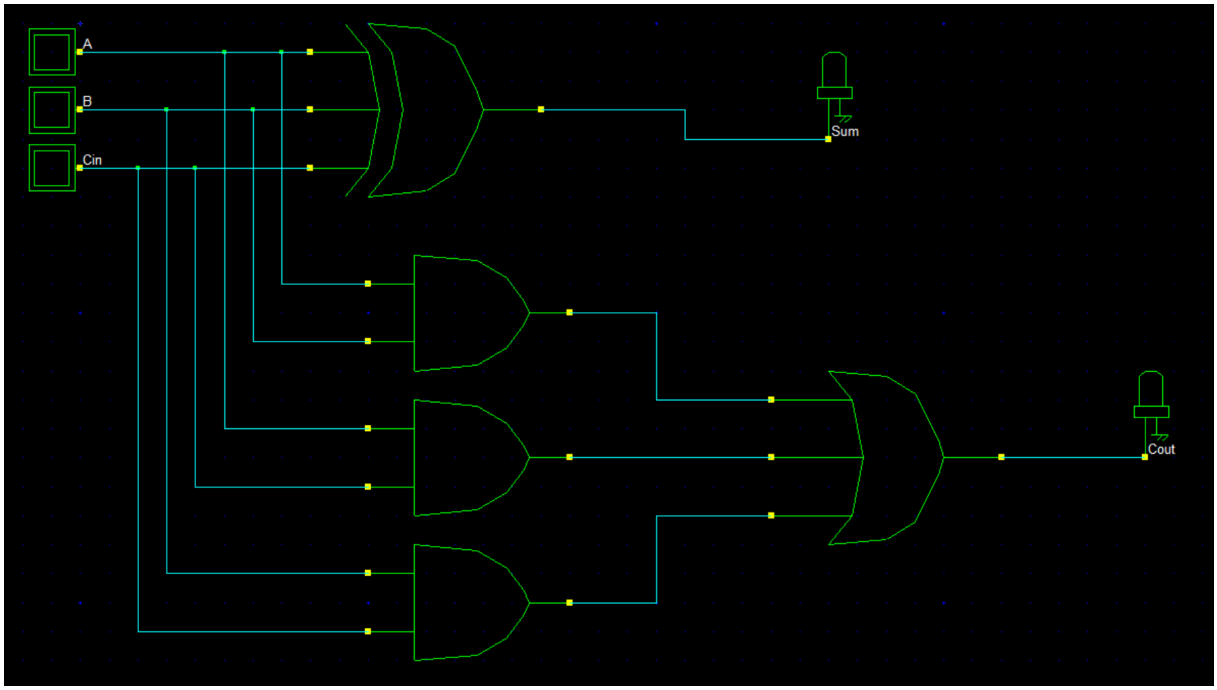
Architecture

The ALU is built using four 1-bit Full Adders, logic blocks, XOR-based subtraction circuitry and four 4:1 multiplexers.

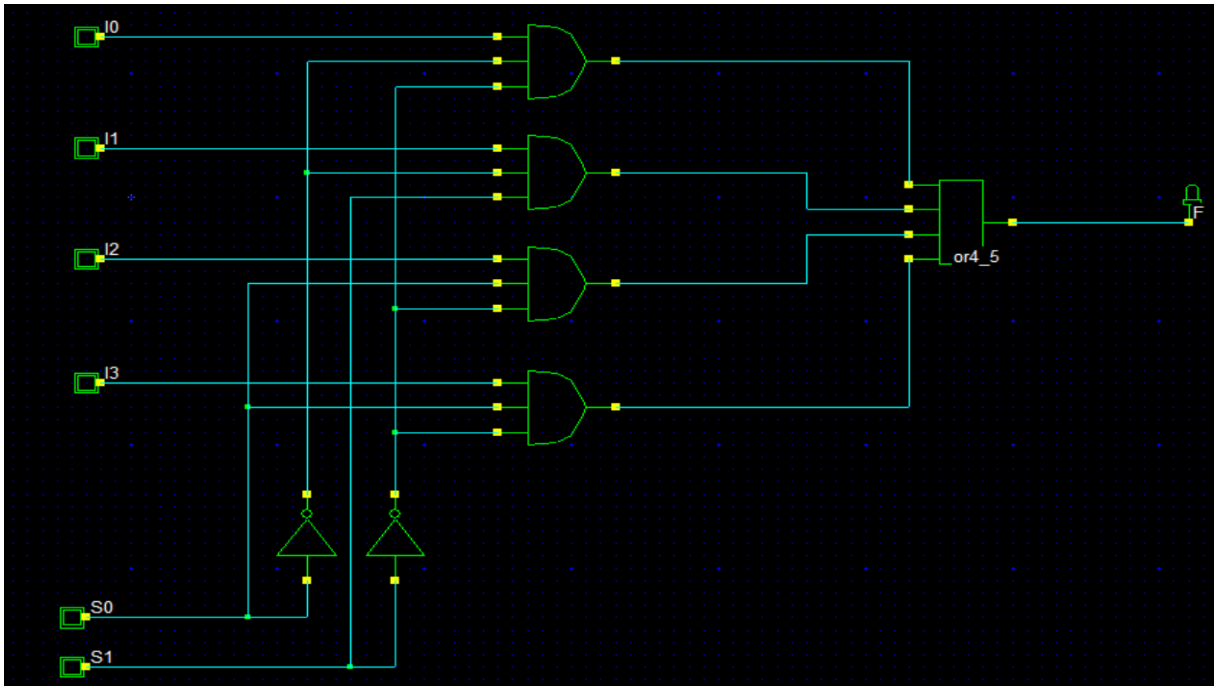
Complete 4-Bit ALU Circuit



1-Bit Full Adder Design



4x1 Multiplexer Design



Operation Selection

00 → Addition/Subtraction
01 → AND
10 → OR
11 → XOR

Results and Verification

The ALU was tested for arithmetic and logical operations. The ripple-carry adder correctly propagates carries, while XOR gates enable two's complement subtraction. Multiplexers successfully select the requested operation.

Generated Verilog HDL

```
// Cleaned DSCH-generated 4-bit ALU
// Fixed module name and removed non-synthesizable simulation code

module FourBitALU(
    Sub,B0,A0,A3,B3,B1,A1,A2,B2,S0,S1,
    F3,F0,F1,Cout,F2
);
 input Sub,B0,A0,A3,B3,B1,A1,A2;
 input B2,S0,S1;
 output F3,F0,F1,Cout,F2;

 wire w7,w9,w13,w15,w17,w18,w19,w21;
 wire w22,w24,w25,w26,w27,w28,w30,w31;
 wire w32,w34,w35,w36,w38,w39,w40,w41;
 wire w42,w43,w44,w45,w46,w47,w48,w49;
 wire w50,w51,w52,w53,w54,w55,w56,w57;
 wire w58,w59,w60,w61,w62,w63,w64,w65;
 wire w66,w67,w68,w69,w70,w71,w72,w73;
 wire w74,w75,w76;

 xor #(3) xor2_1(w7,Sub,B3);
 xor #(3) xor2_2(w9,Sub,B1);
 xor #(2) xor2_3(w13,A3,B3);
 xor #(3) xor2_4(w15,Sub,B0);
 xor #(3) xor2_5(w17,Sub,B2);
 or  #(2) or2_6(w27,A3,B3);
 and #(2) and2_7(w28,B3,A3);
 xor #(2) xor2_8(w30,A2,B2);
 or  #(2) or2_9(w31,A2,B2);
 and #(2) and2_10(w32,B2,A2);
 xor #(2) xor2_11(w34,A1,B1);
 or  #(2) or2_12(w35,A1,B1);
 and #(2) and2_13(w36,B1,A1);
 xor #(2) xor2_14(w38,A0,B0);
 or  #(2) or2_15(w39,A0,B0);
 and #(2) and2_16(w40,B0,A0);

 xor #(1) xor3_1_17(w19,A3,w7,w18);
 and #(1) and2_2_18(w41,w7,A3);
 and #(1) and2_3_19(w42,w18,A3);
 and #(1) and2_4_20(w43,w18,w7);
 or  #(2) or3_5_21(Cout,w41,w42,w43);

 xor #(1) xor3_1_22(w22,A2,w17,w21);
 and #(1) and2_2_23(w44,w17,A2);
 and #(1) and2_3_24(w45,w21,A2);
 and #(1) and2_4_25(w46,w21,w17);
 or  #(2) or3_5_26(w18,w44,w45,w46);

 xor #(1) xor3_1_27(w25,A1,w9,w24);
 and #(1) and2_2_28(w47,w9,A1);
 and #(1) and2_3_29(w48,w24,A1);
 and #(1) and2_4_30(w49,w24,w9);
 or  #(2) or3_5_31(w21,w47,w48,w49);

 xor #(1) xor3_1_32(w26,A0,w15,Sub);
 and #(1) and2_2_33(w50,w15,A0);
 and #(1) and2_3_34(w51,Sub,A0);
 and #(1) and2_4_35(w52,Sub,w15);
 or  #(2) or3_5_36(w24,w50,w51,w52);

 and #(1) and3_1_37(w55,w26,w53,w54);
 and #(1) and3_2_38(w56,w40,w53,S1);
 and #(1) and3_3_39(w57,w39,S0,w54);
 and #(1) and3_4_40(w58,w38,S0,w54);
 or  #(2) or4_5_41(F0,w55,w56,w57,w58);
 not #(1) inv_6_42(w53,S0);
 not #(2) inv_7_43(w54,S1);

 and #(1) and3_1_44(w61,w19,w59,w60);
 and #(1) and3_2_45(w62,w28,w59,S1);
 and #(1) and3_3_46(w63,w27,S0,w60);
 and #(1) and3_4_47(w64,w13,S0,w60);
 or  #(2) or4_5_48(F3,w61,w62,w63,w64);
```

```
 not #(1) inv_6_49(w59,S0);
 not #(2) inv_7_50(w60,S1);

 and #(1) and3_1_51(w67,w22,w65,w66);
 and #(1) and3_2_52(w68,w32,w65,S1);
 and #(1) and3_3_53(w69,w31,S0,w66);
 and #(1) and3_4_54(w70,w30,S0,w66);
 or  #(2) or4_5_55(F2,w67,w68,w69,w70);
 not #(1) inv_6_56(w65,S0);
 not #(2) inv_7_57(w66,S1);

 and #(1) and3_1_58(w73,w25,w71,w72);
 and #(1) and3_2_59(w74,w36,w71,S1);
 and #(1) and3_3_60(w75,w35,S0,w72);
 and #(1) and3_4_61(w76,w34,S0,w72);
 or  #(2) or4_5_62(F1,w73,w74,w75,w76);
 not #(1) inv_6_63(w71,S0);
 not #(2) inv_7_64(w72,S1);

endmodule
```

Conclusion

The project successfully demonstrates the design and implementation of a 4-bit ALU using DSCH and Verilog HDL. The design follows modular RTL principles and serves as a strong foundation for VLSI Front-End and RTL Design roles.